Informe de Avance 1: PINN para resolver ecuaciones diferenciales

Jorge Ruiz de Viñaspre

17 de abril de 2026

1. Introducción

El objetivo del proyecto radica en implementar redes informadas por física (PINN's) para resolver ecuaciones diferenciales (DE), tomando como guía y principal referencia la publicación de Raissi et al. (2019).

Las PINN's se presentan como redes neuronales profundas con funciones de activación diferenciables. Aprovechando el teorema de aproximación universal, el entrenamiento de estas redes se condiciona a una función de pérdida que contiene la ecuación diferencial (residuo físico), sus condiciones iniciales y de borde. Este informe documenta la fase inicial de implementación, centrada en un *proof of concept* para el oscilador armónico y la revisión de arquitecturas continuas y discretas.

2. Objetivos

Los objetivos planteados para esta primera fase han sido:

- Identificar literatura reciente sobre parámetros generales de las redes PINN.
- Estudiar comprensivamente la fuente principal de Raissi et al. (2019) para identificar conceptos relevantes.
- Realizar un *proof of concept* 1D para familiarizarse con la metodología, con el fin de llevarla a contextos más complejos en etapas siguientes del proytecto.
- Identificar dificultades y puntos clave para el desarrollo de las PINN's.
- Centralizar el avance en un repositorio de GitHub actualizable.

3. Metodología y Procedimiento

Se inició con una búsqueda del estado del arte, destacando el trabajo de Zhang et al. (2022) sobre sistemas con dos grados de libertad. El estudio de conceptos se centró en temas como optimizadores, arquitecturas, diferenciación automática y el uso de librerías como PyTorch y aceleraciones por Hardware como CUDA.

3.1. Esquemas de Resolución

Se implementaron dos esquemas principales propuestos de la literatura (Raissi et al., 2019):

1. **Tiempo Continuo:** La red toma un vector de tiempo como dominio de entrenamiento, este es el *input* de la red, el *output* ressultante es un vector de largo igual al input, conteniendo los valores de la función a lo largo del tiempo de input. La pérdida se define como $MSE = MSE_u + MSE_f$, siendo MSE_f el cuadrado del residuo de la ED y MSE_u el error correspondiente a las condiciones de borde e iniciales.

2. **Tiempo Discreto:** La metodología de tiempo discreto integra esquemas clásicos de Runge-Kutta dentro de la arquitectura de la red para avanzar la solución en pasos temporales específicos. El output pasa a ser un vector de 4 (para $q = 2$) dimensiones, conteniendo los estados intermedios de cada step temporal, complementando el sistema RK para llegar a órdenes q teóricamente ilimitados.

3.2. Modelos de Prueba

A lo largo de distintas iteraciones se fue variando el tamaño de la red (ancho y profundidad), como el optimizador y las *epochs*.

Modelo	**Modelo 1**	**Modelo 1.2**	**Modelo 2**	**Modelo 3 (Discreto)**
Deep Layers	(20,20)x1	(20,20)x4	(50,100); (100,100)x3; (100,50)	(32,32)x1
Activación	Tanh()	Tanh()	Tanh()	Tanh()
Optimizador	Adam	Adam	L-BFGS	L-BFGS
Epochs	50.000	20.000	<1000	100 por step

Tabla 1: Resumen de arquitecturas de red implementadas.

4. Resultados y Análisis

La metodología general muestra tener resultados en principio. Los modelos 1, 1.2 y 2 muestran mayormente converger a la solución dentro del dominio de entrenamiento, como se puede apreciar en la siguiente figura

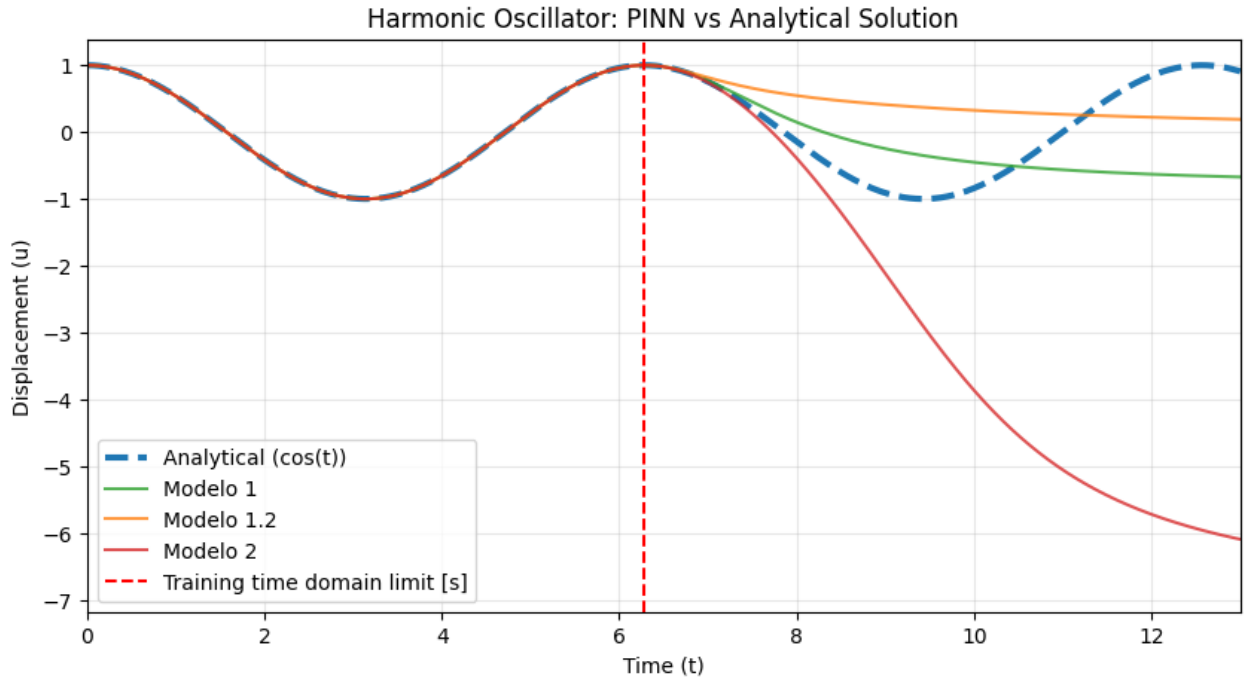

Figura 1: Gráfico de resultados para los modelos de tiempo continuo. La franja vertical denota el término del dominio temporal utilizado para el entrenamiento de los modelos

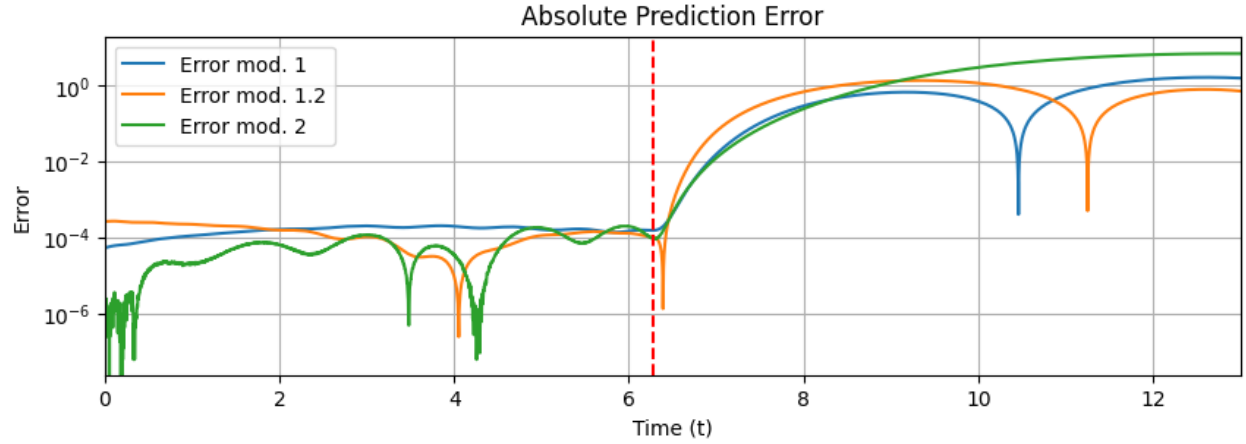

Figura 2: Errores absolutos de las predicciones de los modelos

Como puntos relevantes se encuentra las diferencias entre los tiempos de entrenamiento 5, siendo el modelo 2 el más rápido entre los continuos, obteniendo un error del orden de 10^{-5} en torno a las 600 epochs, versus las versiones 1 y 1.2, cuyos errores lograron bajar del valor dicho, después de miles de epochs.

Además se tiene el resultado de la implementación del modelo de tiempo discreto, el cual muestra errores comparables al sistema continuo para tiempos de entrenamiento inferiores:

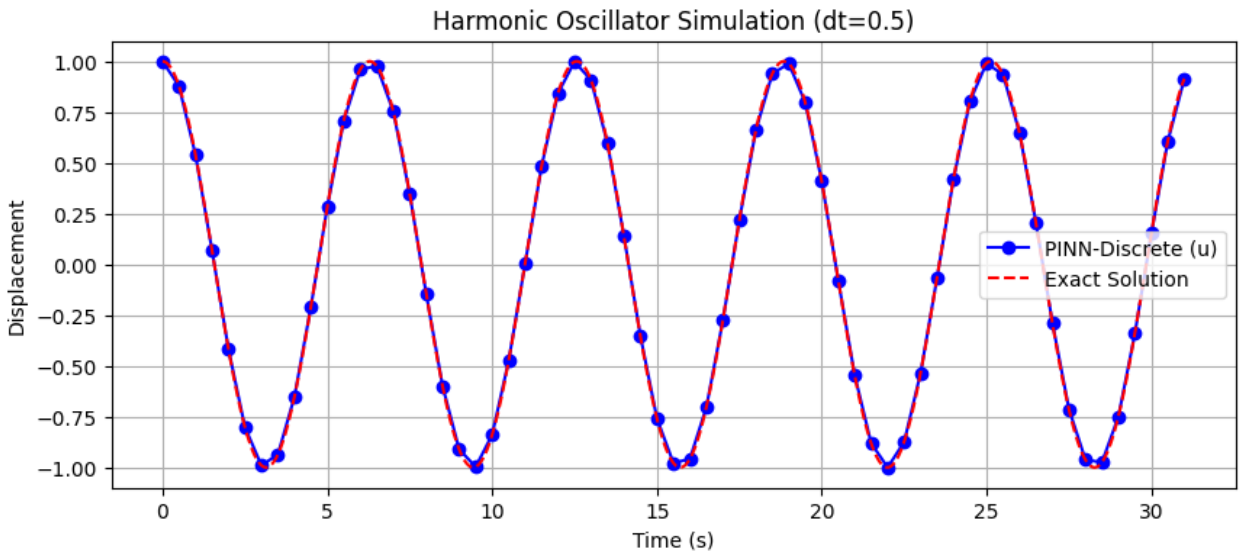

Figura 3: Resultado de modelo de tiempo discreto

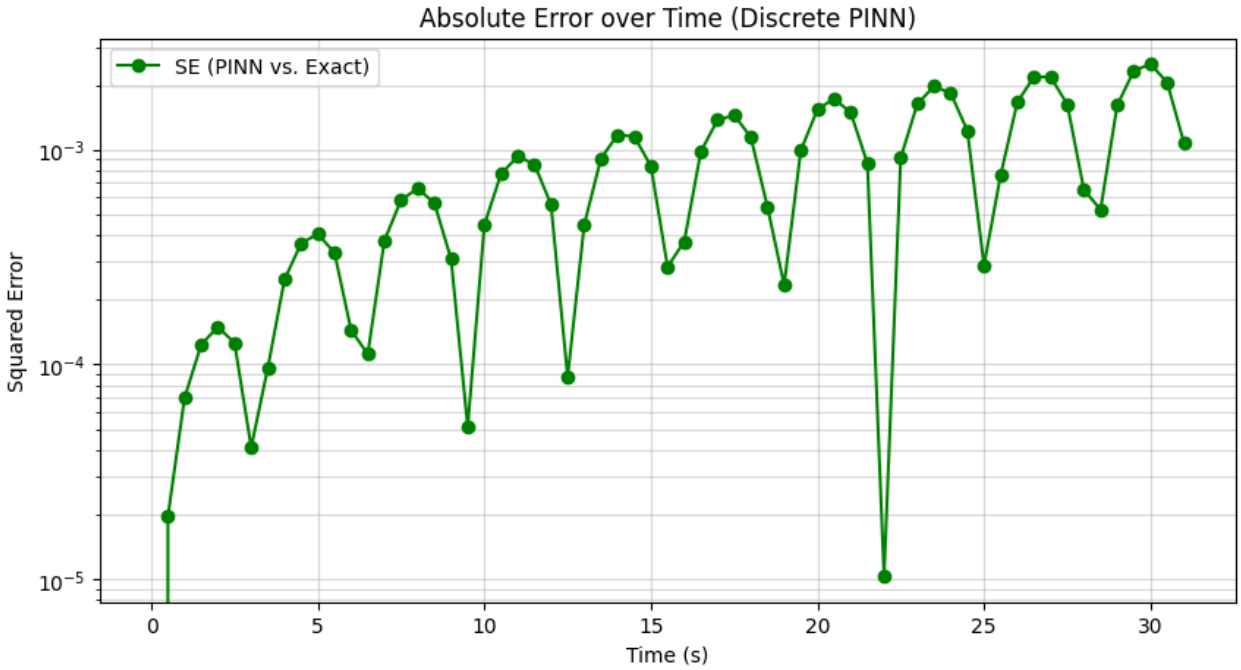

Figura 4: Error absoluto del modelo discreto

Las principales observaciones relevantes consisten en:

1. **Optimizadores:** Adam presenta dificultades para bajar de 10^{-3} en el oscilador armónico, mientras que L-BFGS alcanza precisiones de 10^{-7} rápidamente, validando lo expuesto por Raissi et al. (2019).

2. L-BFGS muestra tener tiempos de convergencia a errores bajos en menores epochs que Adam, mostrando tener resultados similares en error fracción de las epochs.

3. **Oscilación de LossFN usando ADAM** Revisando las tendencias de la Loss function para los modelos que usan Adam y altos números de epochs 5, se nota una tendencia oscilatoria, la cual puede deberse a un learning rate demasiado alto para ese rango de epochs.

4. **Extrapolación:** Los modelos continuos fallan al predecir fuera del rango de entrenamiento.

5. Objetivos para la siguiente Fase

- Análisis a profundidad de la razón de fallos de extrapolación temporal vistos en los modelos continuos. Analizar a qué se debe dicha falla y si es compensable

- Ahondar el estudio y comparación de los modelos discretos, buscando analizar su estabilidad ante extrapolaciones del dominio de entrenamiento.

- Realizar pruebas con schedulers para Adam, buscando amortiguar las oscilaciones detectadas en las lOss functions por medio de un Learning rate progresivamente más fino.

- Viendo la rapidés de L-BFGS, se procederá a llevar a cabo implementaciones con este optimizador para dominio discreto y "continuo" para ED's más complejas, buscando replicar los resultados de los papers. Ejemplos objetivo: Cantilever Beam, Ecuación de Burgers, Schrödinger no lineal.

- Llevar a cabo una comparativa entre los modelos propuestos desde ambas fuentes utilizadas, buscando obtener una comprensión más general de la versatilidad de las PINN's ante distintos contextos operacionales.

Referencias

[1] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, *378*, 686–707. `https://doi.org/10.1016/j.jcp.2018.10.045`

[2] Zhang, W., Zhang, Z., Chen, J., & Zhong, Y. (2022). A general method for solving differential equations of motion using physics-informed neural networks. *Nonlinear Dynamics*, *107*(3), 2363–2382. https://www.mdpi.com/2076-3417/14/17/7694

5.1. Anexo: imágenes

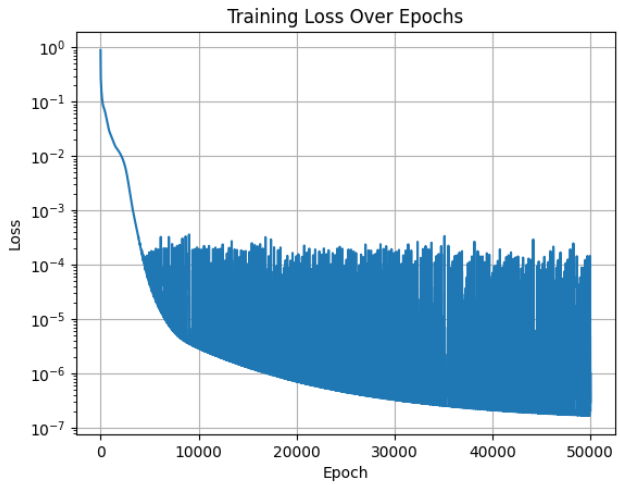

Figura 5: Modelo 1: Función de pérdida a lo largo del entrenamiento

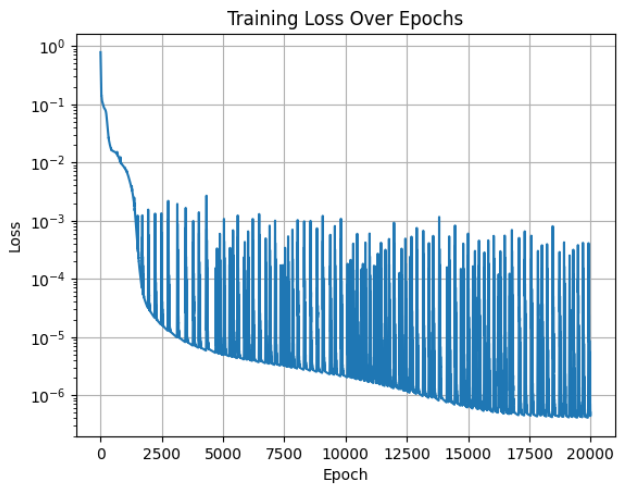

Figura 6: Modelo 1.2: Función de pérdida a lo largo del entrenamiento

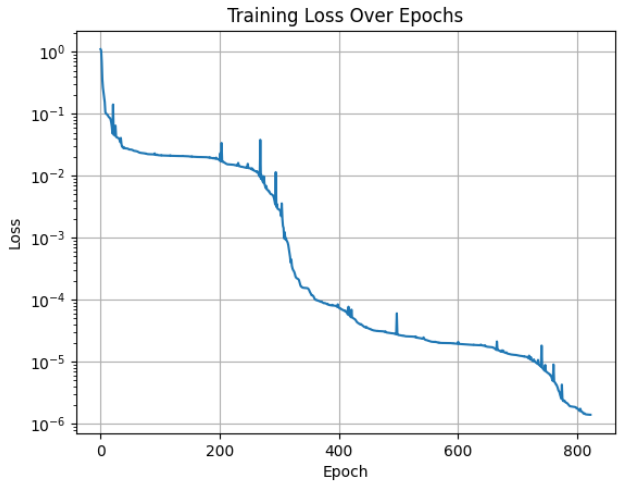

Figura 7: Modelo 2: Función de pérdida a lo largo del entrenamiento